Review 3

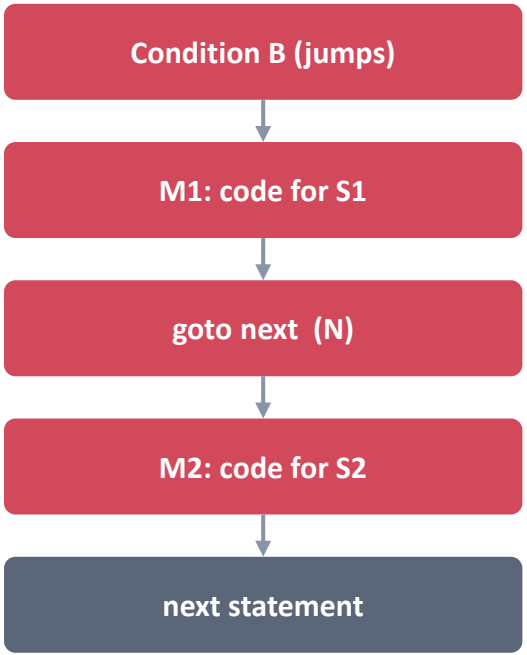
If-else translation, testing, demo and future scope

100% of the implementation

In this part

1. If and if-else translation
2. The parseIf code
3. Worked example
4. Nested if, dangling else
5. Chaining statements, HALT
6. Running the TAC
7. Errors
8. Test suite
9. Demo
10. Contribution and outcome
11. Future scope

If-else translation: where the jumps go



true -> M1, false -> M2

waits in nextlist

S1, N and S2 lists land here

if (B) S1

```
backpatch(B.truelist, M1)
nextlist = B.falselist + S1.nextlist
```

A false condition skips S1 and leaves the statement.

if (B) S1 else S2

```
backpatch(B.truelist,  M1)
backpatch(B.falselist, M2)
nextlist = S1.next + {N} + S2.next
```

N is the goto that jumps over S2 after S1.

The parseIf code

full_implementation.cpp: parseIf, after the condition (condensed)

```
int m1 = cg.nextQuad();                  // M1
StmtAttr s1 = parseStatement();
if (peek().type == TokenType::ELSE) {
    int n = cg.nextQuad();
    cg.emit("goto", "", "", "-1");       // N
    expect(TokenType::ELSE);
    int m2 = cg.nextQuad();              // M2
    StmtAttr s2 = parseStatement();

    cg.backpatch(b.truelist,  m1);
    cg.backpatch(b.falselist, m2);
    res.nextlist = cg.merge(s1.nextlist, cg.makelist(n));
    res.nextlist = cg.merge(res.nextlist, s2.nextlist);
} else {
    cg.backpatch(b.truelist, m1);
    res.nextlist = cg.merge(b.falselist, s1.nextlist);
}
```

1. **Remember M1**
   The quad number where S1 starts.
2. **Emit goto N**
   Only if an else follows. It skips S2.
3. **Backpatch both**
   True goes to M1, false goes to M2.
4. **Return nextlist**
   Unfinished exits of the whole if.

Worked example

Source

```
if (a < b || c == 10) {
    x = a + b * 2;
} else {
    x = 0;
}
```

Quad 1 exists because a failed a < b must still go somewhere: it tests c == 10.

Every target number was filled in later by backpatch().

Generated TAC

```
0: if a < b goto 4      // true: then part
1: goto 2               // false: test c
2: if c == 10 goto 4    // true: then part
3: goto 8               // false: else part
4: t1 = b * 2           // then part (M1)
5: t2 = a + t1
6: x = t2
7: goto 9               // skip else part (N)
8: x = 0                // else part (M2)
9: HALT                 // end of program
```

Nested if and the dangling else

Source

```
if (a > b)
    if (c > d)
        x = 1;
    else
        x = 2;
print x;
```

- Inner false (quad 3) goes to the else at 6.
- Outer false (quad 1) skips to 7.
- Quad 5 jumps over the else once x = 1 is done.

Generated TAC

```
0: if a > b goto 2      // outer true
1: goto 7               // outer false: leave
2: if c > d goto 4      // inner true
3: goto 6               // inner false: inner else
4: x = 1
5: goto 7               // skip inner else
6: x = 2                // inner else
7: print x
8: HALT
```

One extra jump, one extra list: nesting needs no new rule.

Running the TAC to check it

full_implementation.cpp: TACInterpreter::execute (condensed)

```
int pc = 0;
while (pc >= 0 && pc < code.size()) {
    const Quad& q = code[pc];
    if (q.op == "HALT") break;
    else if (q.op == "goto")
        pc = stoi(q.target);
    else if (q.op == "if") {
        // compare left and right with the relop
        if (conditionMet) pc = stoi(q.target);
        else              pc++;
    }
    ...  // =, + - * /, print: do the work, pc++
}
```

Worked example, with a = 3, b = 5, c = 10

```
[OUTPUT]: 13
Execution completed
(10 instructions executed).
```

- Checks the translation end to end
- Use before assign is reported
- Compiler stays single-pass

Error handling

Syntax: missing semicolon

```
x = a + 2  y = 1;
```

```
Syntax Error at line 3, col 1:
Expected ';' but found 'y'
```

Lexical: half an operator

```
if (a & 1) ...
```

```
Lexical Error at line 2, col 7:
Incomplete operator '&'.
Expected '&&'.
```

Lexical: illegal character

```
x = a @ b;
```

```
Lexical Error: Unexpected character
'@' at line 1, col 7
```

Runtime: use before assign

```
if (a < b) x = 1;
```

```
Runtime Error: Variable 'a' used
before being assigned.
```

Demo: if-else

Input program

```
a = 3; b = 5; c = 10;
if (a < b || c == 10) {
    x = a + b * 2;
} else {
    x = 0;
}
print x;
```

Expected: 14 TAC lines, then [OUTPUT]: 13.

```
================ THREE-ADDRESS CODE (TAC) ================
0:      a = 3
1:      b = 5
2:      c = 10
3:      t1 = b * 2
4:      t2 = a + t1
5:      x = t2
6:      if a < b goto 10
7:      goto 8
8:      if c == 10 goto 10
9:      goto 11
10:     x = 1
11:     print x
12:     HALT
==========================================================
```

Demo: nested if and dangling else

Input program

```
a = 5; b = 2; c = 1; d = 4;
if (a > b)
    if (c > d)
        x = 1;
    else
        x = 2;
print x;
```

Expected: 13 TAC lines, then [OUTPUT]: 2, since the else goes with the inner if.

```
================ THREE-ADDRESS CODE (TAC) ================
0:      a = 5
1:      b = 2
2:      c = 1
3:      d = 4
4:      if a > b goto 6
5:      goto 11
6:      if c > d goto 8
7:      goto 10
8:      x = 1
9:      goto 11
10:     x = 2
11:     print x
12:     HALT
==========================================================
```

Conclusion

What we built

- Lexer and recursive descent parser
- Single-pass TAC with backpatching
- Short-circuit ||, && and !
- If and if-else, nested, with correct dangling else
- TAC interpreter and clear error messages

Future scope

1 Optimization of the generated TAC

During the pass (still single-pass)

- Constant folding: ExprAttr already tracks isConst and constVal
- Write a result straight into x, not via a temporary

After the pass (peephole on the flat quads)

- Remove a goto to the next line
- Invert a branch to drop a goto
- Thread jumps that land on another goto

Loops

while, break and continue already work in the code as an extension of the same lists. for loops are next.

Types and declarations

Declare variables and check types.

Arrays, strings, functions

Grow the language beyond integers.

Back end

Register allocation and machine code for a real target.

Future scope: what optimization would give

Now: 10 instructions (real output)

```
0: if a < b goto 4
1: goto 2
2: if c == 10 goto 4
3: goto 8
4: t1 = b * 2
5: t2 = a + t1
6: x = t2
7: goto 9
8: x = 0
9: HALT
```



Projected: 7 instructions

```
      0: if a < b goto L1
      1: if c != 10 goto L2
L1:   2: t1 = b * 2
      3: x = a + t1
      4: goto L3
L2:   5: x = 0
L3:   6: HALT
```

30%

fewer instructions, projected for this example

How we would check it: run the original and the optimized TAC in the same interpreter and require the same [OUTPUT] and the same final variables.

Individual contribution

Module	What was built	Individual
Lexer	Tokens, line and column tracking, comments, malformed-token errors	SHASHANK MITALI
Recursive descent parser	Grammar, precedence, dangling else	AARUSHI
TAC engine	Quads, temporaries, printer	SHASHANK
Backpatching and attributes	makelist, merge, backpatch, ExprAttr, BoolAttr, StmtAttr	MITALI
Short-circuit booleans	||, && and ! with true and false lists	AARUSHI
If and if-else translation	Markers, nextlist, statement chaining, HALT	SHASHANK
TAC interpreter and errors	Virtual machine, jump validation, error messages	MITALI
Tests and demo	Nine test programs, demo runs	AARUSHI

Code: github.com/skifv-aarushi/cd_project

References

1. Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson, 2006.

2. Cooper, Torczon. Engineering a Compiler, 3rd ed. Morgan Kaufmann, 2022.

3. Project code: github.com/skifv-aarushi/cd_project